Chapter 4

Communication

Contents

- Layer protocols
- Types of Communication
- Remote Procedure Call
- Remote Object invocation

01

Communication

- **Interprocess communication** is at the heart of all distributed systems.
- It makes no sense to study distributed systems without carefully examining the ways that processes on different machines can exchange information.
- Communication in distributed systems is always based on low-level message passing as offered by the underlying network(TCP, UDP, Socket).
- Modern distributed systems often consist of thousands or even millions of processes scattered across a network with unreliable communication such as the Internet.

Con'd

- The rules that communicating processes must adhere to, known as **protocols,** and concentrate on structuring those protocols in the form of layers.
- A **protocol** is a set of rules that define:
- How messages are sent
- How messages are formatted
- How processes understand each other

👉 Example:

✓ HTTP defines how a client and server talk

✓ WebSocket defines real-time communication rules

- Three widely-used models for communication:
  - ✓ Remote Procedure Call (RPC),
  - ✓ Message-Oriented Middleware (MOM), and
  - ✓ Data streaming.

Fundamentals

- Our first model for communication in distributed systems is the remote procedure call (RPC).
- **An RPC** aims at hiding most of the details of message passing, and is ideal for client-server applications.
- In many distributed applications, communication does not follow the rather strict pattern of client-server interaction. However, the low-level communication (UDP/TCP) facilities of computer networks are in many ways not suitable due to their lack of distribution transparency.
- An alternative is to use a high-level message-queuing model (MOM), in which communication proceeds much the same as in electronic mail systems.

Con'd

- Message-oriented middleware (MOM) is a software infrastructure that enables applications, services, or components to communicate by exchanging discrete messages through an intermediary message broker (or sometimes broker less). Its core promise is decoupling the producer of a message from its consumer(s) in time, space, and technology.

05

Fundamental Principles of MOM

- **Asynchronous communication** – The sender does not wait for the receiver.
- **Loose coupling** – Producer and consumer do not need to know each other's location, implementation language, or even be running at the same time.
- **Message persistence** – Messages can survive, crashes and restarts.
- **Delivery semantics** – Usually at-least-once or exactly-once; sometimes at-most-once.

Layered protocols

- Due to the absence of shared memory, all communication in distributed systems is based on sending and receiving (low level) messages.
- When process A wants to communicate with process B, it first builds a message in its own address space.
- Then it executes a system call that causes the operating system to send the message over the network to B.
- Although this basic idea sounds simple enough, in order to prevent chaos, A and B have to agree on the meaning of the bits being sent.

Con'd

- The OSI model is designed to allow open systems to communicate.
- An open system is one that is prepared to communicate with any other open system by using standard rules that govern the format, contents, and meaning of the messages sent and received.
- These rules are formalized in what are called protocols.
- To allow a group of computers to communicate over a network, they must all agree on the protocols to be used.
- A distinction is made between two general types of protocols.

Con'd

- With connection-oriented protocols, before exchanging data the sender and receiver first explicitly establish a connection, and possibly negotiate the protocol they will use.
- When they are done, they must release (terminate) the connection.
- The telephone is a connection-oriented communication system.
- With connectionless protocols, no setup in advance is needed.
- The sender just transmits the first message when it is ready.
- Dropping a letter in a mailbox is an example of connectionless communication. With computers, both connection-oriented and connectionless communication are common.

Con'd

- In the OSI model, communication is divided up into seven levels or layers.
- Each layer deals with one specific aspect of the communication.
- In this way, the problem can be divided up into manageable pieces, each of which can be solved independent of the others.
- Each layer provides an interface to the one above it.
- The interface consists of a set of operations that together define the service the layer is prepared to offer its users.

Con'd

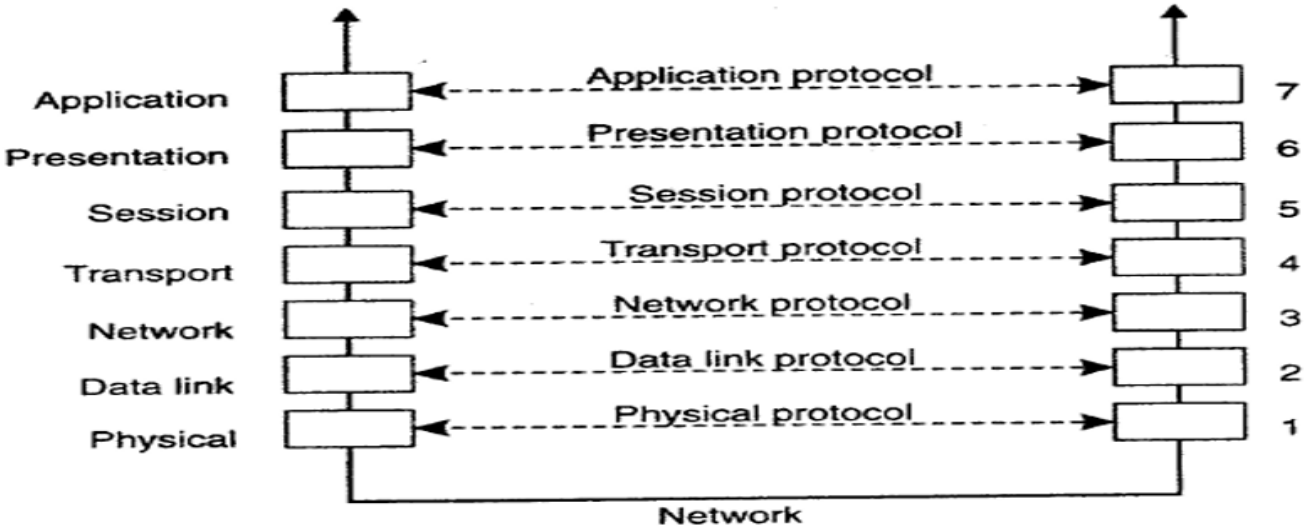

Figure 4-1. Layers, interfaces, and protocols in the OSI model.

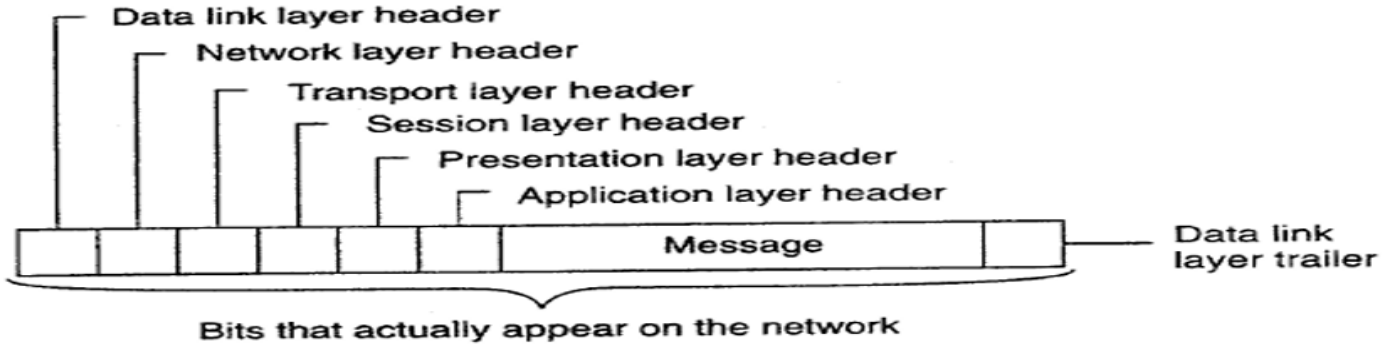

Figure 4-2. A typical message as it appears on the network.

REMOTE PROCEDURE CALL

- Many distributed systems have been based on explicit message exchange between processes.
- However, the procedures send and receive do not conceal communication at all, which is important to achieve access transparency in distributed systems.
- When a process on machine A calls' a procedure on machine B, the calling process on A is suspended, and execution of the called procedure takes place on B. Information can be transported from the caller to the callee in the parameters and can come back in the procedure result.
- No message passing at all is visible to the programmer. This method is known as Remote Procedure Call, or often just RPC.

Con'd

- Remote Procedure Call (RPC) is a protocol that one program can use to request a service from a program located in another computer on a network without having to understand the network's details.
- A procedure call is also sometimes known as a function call or a subroutine call.
- A stub in distributed computing is a piece of code used for converting parameters passed during a Remote Procedure Call (RPC).
- The main idea of an RPC is to allow a local computer (client) to remotely call procedures on a remote computer (server).

Con'd

- RPC is a powerful technique for constructing distributed, client-server-based applications. It is based on extending the notion of conventional, or local procedure calling, so that the called procedure need not exist in the same address space as the calling procedure.
- The two processes may be on the same system, or they may be on different systems with a network connecting them.
- By using RPC, programmers of distributed applications avoid the details of the interface with the network.
- RPC separates the application from the details of how data is transmitted over the network. This means the application does not need to know or manage the underlying network protocols or hardware, and it can work with different types of transport methods, such as TCP, UDP, or others, without changing the application code.

Con'd

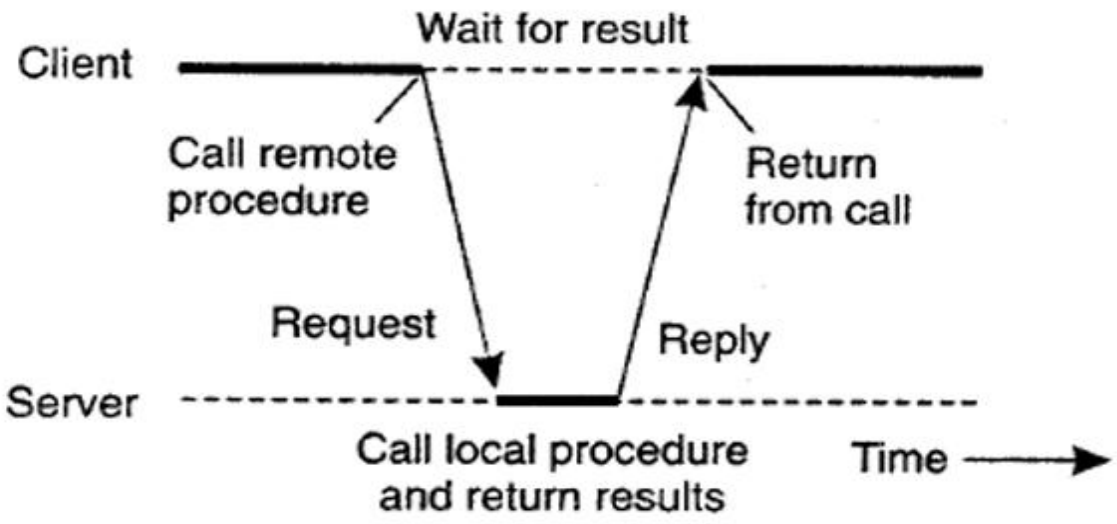

Figure 4-6. Principle of RPC between a client and server program.

Parameter Passing

- The function of the client stub is to take its parameters, pack them into a message, and send them to the server stub.
- Packing parameters into a message is called parameter marshaling.

Con'd

The following steps take place during an RPC:

- 1. A client invokes a client stub procedure, passing parameters in the usual way. The client stub resides within the client's own address space.
- 2. The client stub marshalls the parameters into a message. Marshalling includes converting the representation of the parameters into a standard format, and copying each parameter into the message.
- 3. The client stub passes the message to the transport layer, which sends it to the remote server machine.

Con'd

- 4. On the server, the transport layer passes the message to a server stub, which unmarshalls the parameters and calls the desired server routine using the regular procedure call mechanism.
- 5. When the server procedure completes, it returns to the server stub (e.g., via a normal procedure call return), which marshall the return values into a message. The server stub then hands the message to the transport layer.
- 6. The transport layer sends the result message back to the client transport layer, which hands the message back to the client stub. The client stub unmarshalls the return parameters and execution returns to the caller.

MESSAGE-ORIENTED COMMUNICATION

- Remote procedure calls and remote object invocations contribute to hiding communication in distributed systems, that is, they enhance access transparency.

Message-Oriented Transient Communication

- Many distributed systems and applications are built directly on top of the simple message-oriented model offered by the transport layer.
- To better understand and appreciate the message-oriented systems as part of middleware solutions, we first discuss messaging through transport-level sockets.

Berkeley Sockets

- The transport layer (TCP, UDP, etc.) has many features, but directly using them can be complex.
- A standard interface provides a simple, consistent set of functions (primitives) so programmers don't have to deal with low-level details
- Special attention has been paid to standardizing the interface of the transport layer to allow programmers to make use of its entire suite of (messaging) protocols through a simple set of primitives.
- Also, standard interfaces make it easier to port an application to a different machine.
- Another important interface is XTI, which stands for the X10pen Transport Interface, formerly called the Transport Layer Interface (TLI), and developed by AT&T.

Con'd

- Sockets and XTI are very similar in their model of network programming.
- Conceptually, a socket is a communication end point to which an application can write data that are to be sent out over the underlying network, and from which incoming data can be read.
- A socket forms an abstraction over the actual communication end point that is used by the local operating system for a specific transport protocol.
- In the following text, we concentrate on the socket primitives for TCP, which are shown in Fig. 4-14.

Con'd

Primitive	Meaning
Socket	Create a new communication end point
Bind	Attach a local address to a socket
Listen	Announce willingness to accept connections
Accept	Block caller until a connection request arrives
Connect	Actively attempt to establish a connection
Send	Send some data over the connection
Receive	Receive some data over the connection
Close	Release the connection

Figure 4-14. The socket primitives for TCPIIP.

The Message-Passing Interface (MPI)

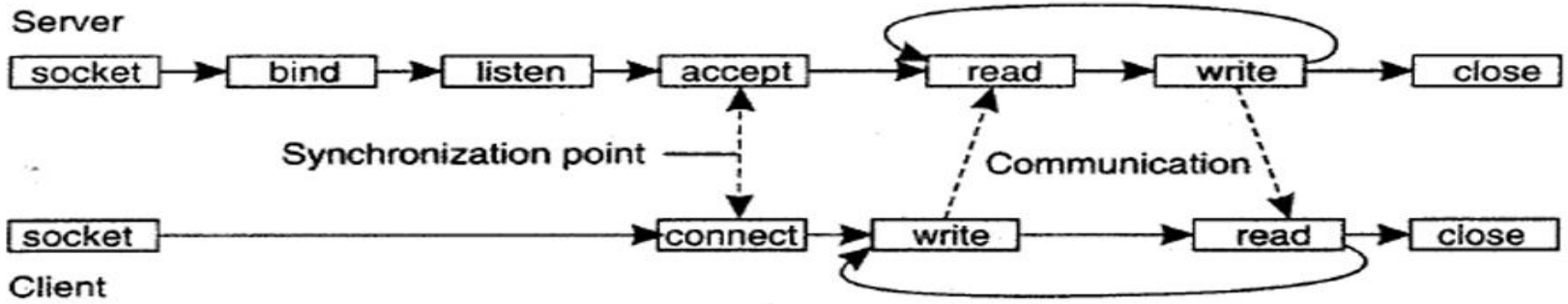

Figure 4-15. Connection-oriented communication pattern using sockets.

Con'd

- Messages can, in principle, contain any data.
- The only important aspect from the perspective of middleware is that messages are properly addressed.
- In practice, addressing is done by providing a system wide unique name of the destination queue.
- In some cases, message size may be limited, although it is also possible that the underlying system takes care of fragmenting and assembling large messages in a way that is completely transparent to applications.

Con'd

Primitive	Meaning
Put	Append a message to a specified queue
Get	Block until the specified queue is nonempty, and remove the first message
Poll	Check a specified queue for messages, and remove the first. Never block
Notify	Install a handler to be called when a message is put into the specified queue

Figure 4-18. Basic interface to a queue in a message-queuing system.

- **Put (Send):** Adds a message to the queue; producer doesn't care who or when it's read.
- **Get (Blocking Receive):** Waits for a message if the queue is empty; process pauses until data arrives.
- **Poll (Non-blocking Receive):** Checks for a message; if none, continues without waiting.
- **Notify (Event-driven):** Uses a callback; system triggers it automatically when a message arrives.

Processes communicate via messages rather than shared memory (as we saw before).

- Put, Get, Poll, Notify define how processes send, receive, or react to messages.
- These primitives operate above the transport/network layers (TCP/IP or OSI) to make inter-process communication (IPC) reliable and flexible.
- They show practical implementation of protocols discussed in the last slide: how data moves and how processes agree on handling it.

Message Brokers

- An important application area of message-queuing (method where messages are placed in a **queue** (like a waiting line) until the receiving application is ready to process them.) systems is integrating existing and new applications into a single, coherent distributed information system.
- Integration requires that applications can understand the messages they receive.
- In practice, this requires the sender to have its outgoing messages in the same format as that of the receiver.
- The problem with this approach is that each time an application is added to the system that requires a separate message format, each potential receiver will have to be adjusted in order to produce that format.
- An alternative is to agree on a common message format, as is done with traditional network protocols. Unfortunately, this approach will generally not work for message-queuing systems.

Con'd

- In message-queuing systems, conversions are handled by special nodes in a queuing network, known as message brokers.
- A message broker acts as an application-level gateway in a message queuing system.
- Its main purpose is to convert incoming messages so that they can be understood by the destination application. Note that to a message-queuing system, a message broker is just another application, as shown in Fig. 4-21.
- In other words, a message broker is generally not considered to be an integral part of the queuing system.

Con'd

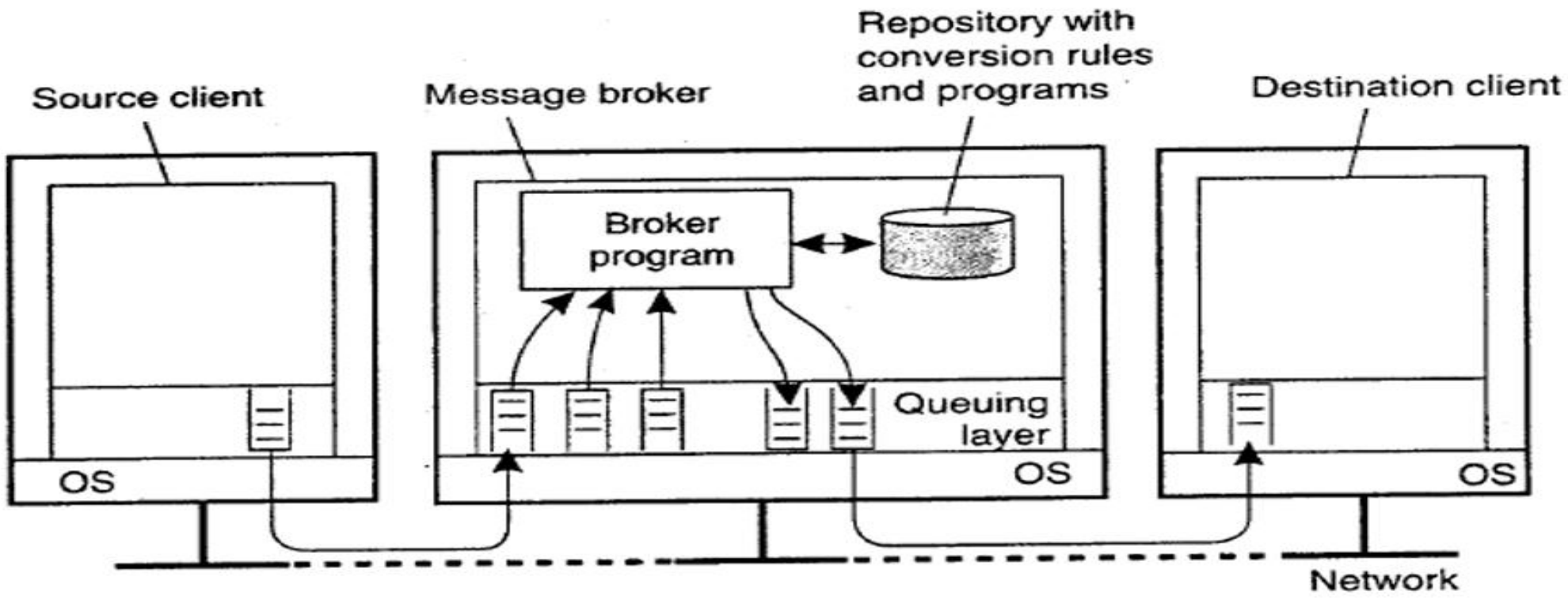

Figure 4-21. The general organization of a message broker in a message-queuing system.

Con'd

- A message broker can be as simple as a re-formatter for messages.
- In a more advanced setting, a message broker may act as an application-level gateway, such as one that handles the conversion between two different database applications.
- In such cases, frequently it cannot be guaranteed that all information contained in the incoming message can actually be transformed into something appropriate for the outgoing message.

STREAM-ORIENTED COMMUNICATION

- In continuous (representation) media, the temporal relationships between different data items are fundamental to correctly interpreting what the data actually means.
- We already gave an example of reproducing a sound wave by playing out an audio stream.
- As another example, consider motion.
- Motion can be represented by a series of images in which successive images must be displayed at a uniform spacing T in time, typically 30-40 ms per image.
- Correct reproduction requires not only showing the stills in the correct order, but also at a constant frequency of images per second.
- In contrast to continuous media, discrete (representation) media, is characterized by the fact that temporal relationships between data items are not fundamental to correctly interpreting the data.
- Typical examples of discrete media include representations of text and still images, but also object code or executable files.

Data Stream

- To capture the exchange of time-dependent information, distributed systems generally provide support for data streams.
- A data stream is nothing but a sequence of data units.
- Data streams can be applied to discrete as well as continuous media.
- For example, UNIX pipes or TCP IP connections are typical examples of (byte-oriented) discrete data streams.
- Playing an audio file typically requires setting up a continuous data stream between the file and the audio device.

Data Stream

- Timing is crucial to continuous data streams.
- To capture timing aspects, a distinction is often made between different transmission modes.
- In asynchronous transmission mode the data items in a stream are transmitted one after the other, but there are no further timing constraints on when transmission of items should take place. This is typically the case for discrete data streams.

Con'd

- Streams can be simple or complex.
- A simple stream consists of only a single sequence of data, whereas a complex stream consists of several related simple streams, called sub-streams.
- The relation between the sub-streams in a complex stream is often also time dependent.
- From a distributed systems perspective, we can distinguish several elements that are needed for supporting streams.
- For simplicity, we concentrate on streaming stored data, as opposed to streaming live data.

Con'd

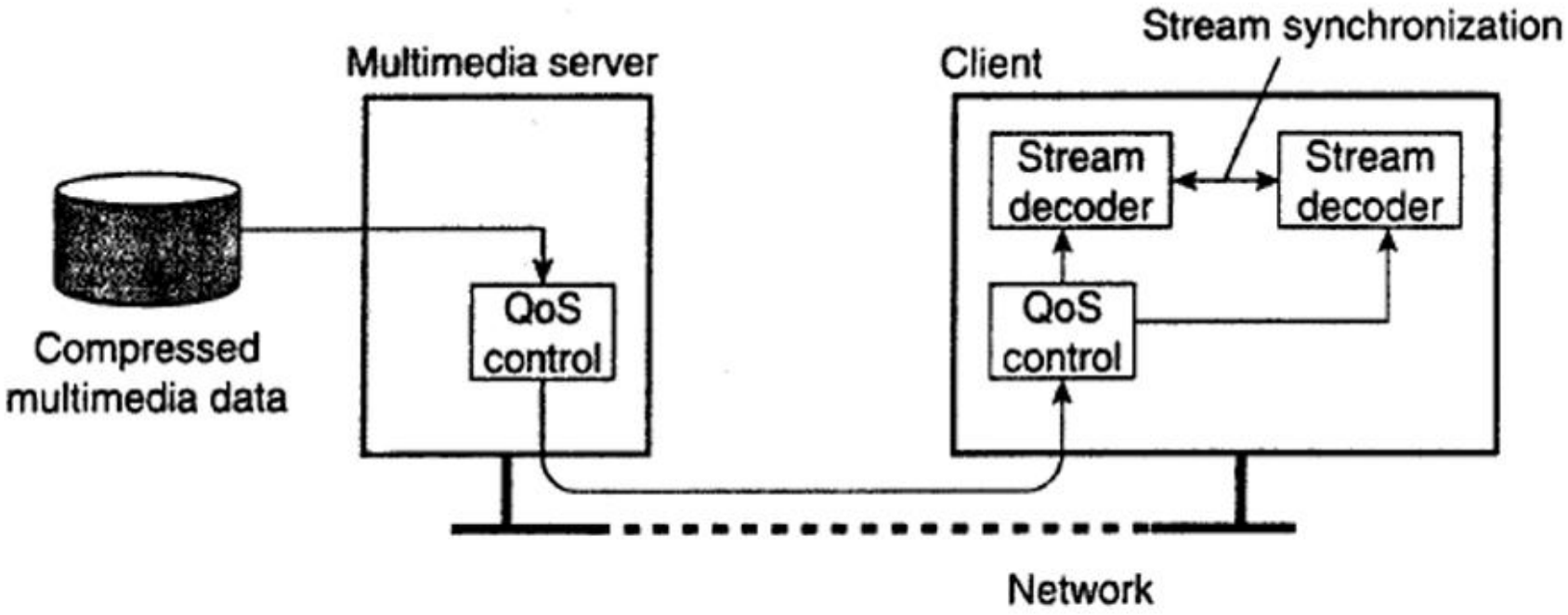

Figure 4-26. A general architecture for streaming stored multimedia data over a network.

Streams and Quality of Service

- Timing (and other nonfunctional) requirements are generally expressed as Quality of Service (QoS) requirements.
- QoS for continuous data streams mainly concerns timeliness, volume, and reliability.
- Main QoS Parameters:
  1) The required bit rate at which data should be transported.
  2) The maximum delay until a session has been set up (i.e., when an application can start sending data).
  3) The maximum end-to-end delay(One-way delay) (i.e., how long it will take until a data unit makes it to a recipient). ss
  4) The maximum delay variance,or jitter.->Packets don't arrive at a constant speed Some are fast, some are slow
  5) The maximum round-trip delay. Sender → Receiver → Back to Sender. Important for **interactive systems(Video call, audio call)**

Stream Synchronization

- An important issue in multimedia systems is that different streams, possibly in the form of a complex stream, are mutually synchronized.
- Synchronization of streams deals with maintaining temporal relations between streams.
- Two types of synchronization occur. The simplest form of synchronization is that between a discrete data stream and a continuous data stream.
- Synchronization takes place at the level of the data units of which a stream is made up.
- In other words, we can synchronize two streams only between data units.

Con'd

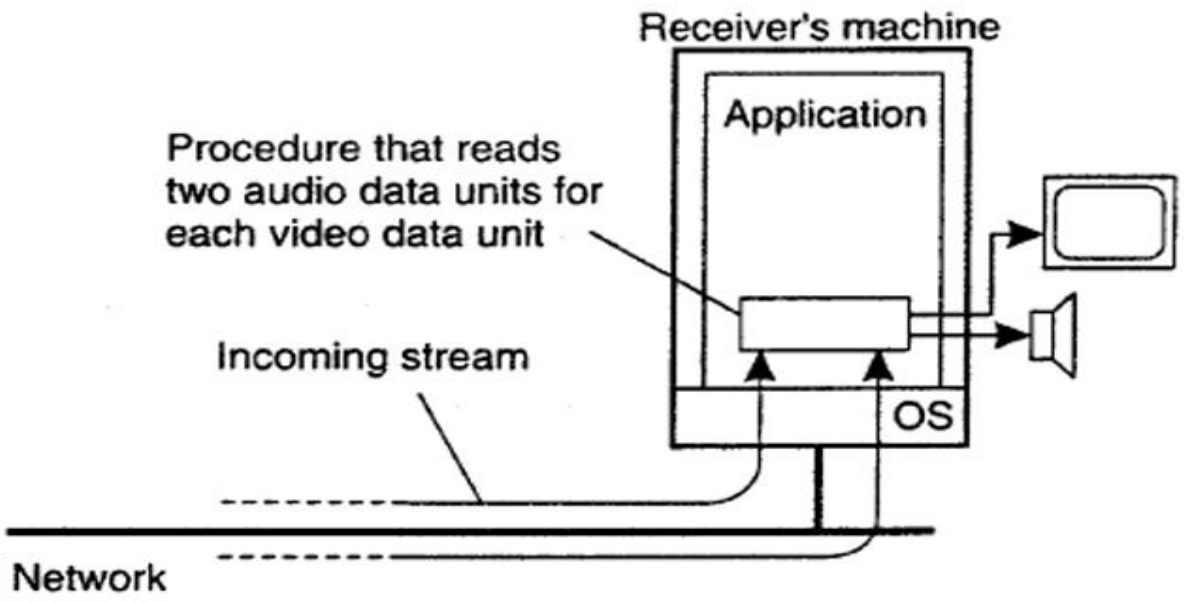

Figure 4-29. The principle of explicit synchronization on the level data units.

MULTICAST COMMUNICATION

- An important topic in communication in distributed systems is the support for sending data to multiple receivers, also known as multicast communication.
- The basic idea in application-level multicasting is that nodes organize into an overlay network, which is then used to disseminate information to its members.
- An important class of communication protocols in distributed systems is multicasting.
- The basic idea is to disseminate information from one sender to multiple receivers.

Con'd

- There are two different approaches.
- First, multicasting can be achieved by setting up a tree from the sender to the receivers.
- Considering that how nodes can self-organize into peer-to-peer system, solutions have also appeared to dynamically set up trees in a decentralized fashion.
- Another important class of dissemination solutions deploys epidemic protocols. These protocols have proven to be very simple, yet extremely robust.
- Apart from merely spreading messages, epidemic protocols can also be efficiently deployed for aggregating information across a large distributed system.

Quiz (5%)

1. What is a protocol and give example(s)? (1pt)
2. What is a message broker and how does it work?. (1pt)
3. What is an open system? (1pt)
4. Discuss simple and complex stream. (support with examples).(1pt)
5. What are the two approaches to disseminate information? (1pt)

Thank You